CivicPulse project summary

Step 1: Define the problem

- Built CivicPulse to monitor Pakistani civic issues from news reports.
- The main target is `n_reports`:
  - the number of classified reports for a city, issue type, and hour.
- The project predicts reports, not confirmed real-world incidents.
- Main issue categories include:
  - Mobility
  - Electricity
  - Water
  - Municipal

Step 2: Collect and document data

- Added FreeNewsAPI for recent Pakistani news.
- Added GDELT Project 2.0 Events for historical data.
- Used GDELT because it provides a large historical event archive.
- Added Open-Meteo as an independent weather-feature source.
- Weather features include:
  - temperature
  - precipitation
  - wind speed
- NDMA was not ingested because its terms restrict reproduction and its API is disallowed by robots.txt.
- Documented:
  - source URLs;
  - licenses;
  - limitations;
  - sampling;
  - provenance rules;
  - source ownership.

Step 3: Build the data pipeline

- Created Bronze, Silver, and Gold data layers.
- Bronze stores raw downloaded files.
- Silver cleans, validates, deduplicates, and standardizes articles/events.
- Gold creates civic issue events and hourly report counts.
- Added data-quality checks and quarantining.
- Added manifest tracking for every raw file:
  - source URL;
  - query;
  - timestamp;

    - SHA-256;
    - file size;
    - license.
- Added Open-Meteo weather files to the manifest and warehouse.
- Joined weather to Gold by:
    - city;
    - UTC hour.
- Kept weather separate from the civic-event target.
- Local warehouse results included approximately:
    - 60,000+ Bronze rows;
    - 13,000+ Silver articles;
    - 140+ Gold civic events;
    - 212,832 weather rows across 12 cities.
- Added tests throughout the pipeline.

Step 4: Model and evaluate

- Locked a chronological cutoff:

```
2026-04-06 18:00 UTC
```

- Used a time-based split:
    - earlier data for training;
    - later data for testing.
- Added the required weekly seasonal-naive baseline.
- The baseline predicts using the same city/issue/hour from 168 hours earlier.
- Created a complete city/issue/hour grid.
- Missing report counts are represented as zero for the declared retrospective evaluation grid.
- Added an explainable Poisson regression model.
- Model features include:
    - city;
    - issue type;
    - hour of day;
    - day of week;
    - temperature;
    - precipitation;
    - wind speed.
- Missing weather values are imputed during preprocessing.
- Selected the Poisson regularization parameter using training data only.
- Added:
    - prediction intervals;
    - grouped error summaries by city and issue type;
    - full-grid MAE;
    - event-bearing MAE.
- Final results:
- `Poisson full-grid MAE: 0.0000690631`

```
Weekly baseline MAE:    0.0000551752
```

- The baseline performed better.
- The Poisson model is technically working, but the dataset is very sparse:
  - only 28 test reports;
  - nearly 1 million full-grid rows;
  - most event rows contain only one report.
- Therefore, the Poisson model predicts a very small value for most rows.
- We made no claim that the complex model improves performance.

Automation and GitHub

- Created an hourly GitHub Actions workflow.
- The workflow:
  - refreshes FreeNewsAPI;
  - fetches recent GDELT data;
  - fetches/restores Open-Meteo weather;
  - rebuilds Bronze, Silver, and Gold;
  - retrains and evaluates the model.
- Fixed GitHub Actions so weather data is available in CI.
- CI now successfully produces:

```
weather_rows: 212832
```

- Used a separate feature branch instead of modifying `main` directly.
- Pushed the completed work to GitHub.
- Merged the completed changes into `main`.
- Verified the pipeline with passing tests.

Current project status:

- Steps 1–4 are complete.
- The pipeline refreshes hourly.
- The model retrains during the hourly workflow.
- Step 5 can now begin: building the RAG/retrieval layer.

Step 5: Retrieve Trusted Information (RAG)

- Added a RAG system to find relevant information from project documents and selected public sources.
- Converts documents into searchable text chunks.
- Retrieves relevant passages for a user's question.
- Keeps source details so answers can link back to references.
- Uses Gold data directly for report counts and city rankings.
- Missing data is shown as unavailable, not as zero reports.

Step 6: Generate Grounded Answers

- Connected retrieved information and CivicPulse results to an OpenRouter-compatible language model.
- The model explains the evidence in simple language.
- Answers include source names and links.
- The chatbot distinguishes news reports from confirmed incidents and model estimates from observed counts.
- If the available evidence does not support an answer, the chatbot explicitly says so.

Step 7: Store and Show Results

- Added Supabase to store Gold report counts and Poisson forecasts.
- The hourly workflow can upsert new results using stable keys.
- Built a Streamlit app to show project information, report data, forecasts, and the chatbot.
- The app reads stored results from Supabase using server-side secrets.
- Users can explore the project without needing to understand the full data pipeline.